

Problemi Tanımak: Sınıflandırma Eksenleri ve Sınırları

Ömer Faruk Yavuz

August 2026

Özet

Bir problemi “hangi sınıfa ait” diye sormak, çözüm aramadan önce yapılan ayrı bir işlemdir; sınıf tanındığında o sınıf için birikmiş yöntemler doğrudan kullanılabilir hale gelir. Ancak bu sınıflandırmanın öncesinde daha belirleyici bir adım vardır: problemin temsili. Gerçek durum önce bir soyut yapıya çevrilir, sınıf ancak o yapı üzerinde belirlenir. Temsil seçimi tarafsız değildir, çünkü hangi çözümlerin görülebilir olduğunu belirler; aynı problem graf olarak temsil edildiğinde başka, kısıt kümesi olarak temsil edildiğinde başka yöntemler aklı gelir. Mühendislikte tek bir resmî problem taksonomisi yoktur. Amaca, yapıya, alana ve hesaplama zorluğuna göre birbirinden bağımsız eksenler vardır ve aynı problem bu eksenlerin hepsinde aynı anda sınıflandırılır; eksenler karıştırıldığında “optimizasyon problemi” ile “NP-zor problem” aynı türden etiket sanılır. Gerçek dünyadan koda giden yol tek adımlı değil katmanlıdır, ve katmanlar arasında akış tek yönlü değildir. Çerçevenin asıl sınırı, sınıflandırmanın yalnızca zaten düzgün tanımlanmış problemlerde işlemesidir; sahadaki problemler bu biçimde gelmez ve asıl emek onları sınıflandırılabilir hale getirmeye gider.

Giriş

“Bu aslında bir çizelgeleme problemi” cümlesi bir çözüm değildir. Problemi bilinen bir kümeye yerleştirme işlemidir ve çözümün hangi alandan aranacağını belirler.

Bu yerleştirmenin değeri pratiktir. Bilinen problem ailelerinin her biri için literatürde birikmiş yöntemler, kanıtlanmış zorluk sonuçları ve hazır yazılım kütüphaneleri vardır. Problem bu ailelerden birine oturduğunda o birikim devralınır. Oturmadığında mühendis kendi yöntemini kurmak zorunda kalır; bu hem pahalıdır hem de üretilen çözümün ne kadar iyi olduğu bilinmez.

Üç kavram burada sık sık birbirinin yerine kullanılır ve ayrılmaları gerekir.

- **Problem temsili** (*problem representation*): Problemin hangi soyut yapıyla ifade edildiği. Örnek: “Bu durumu düğümler ve kenarlar olarak yazıyorum.”
- **Problem sınıfı** (*problem class*): Problemin hangi bilinen aileye ait olduğu. Örnek: “Bu bir en kısa yol problemi.”
- **Problem taksonomisi** (*problem taxonomy*): Sınıfların kendi aralarında nasıl gruplandığını gösteren sınıflandırma sistemi. Örnek: “En kısa yol, graf problemlerinin altındadır.”

Yaygın bir kabul, bu sınıflandırmanın biyolojideki canlı sınıflandırması gibi tek ve kesin olduğudur. Kabul yanlıştır. Mühendislikte tek bir resmî taksonomi yoktur; farklı alanlar farklı eksenler kullanır ve bir problem birden çok eksenle aynı anda yer alır.

Not, bu üç kavramın ne olduğunu, sınıflandırma eksenlerinin nasıl ayrıldığını, gerçek dünyadan koda giden katmanlı yolu ve bu çerçevenin nerede işlemediğini kapsar. Tek tek algoritmaların nasıl çalıştığı kapsam dışındadır.

Problem Temsili

Temsil nedir

Gerçek dünyadaki bir durum, bir algoritmanın işleyebileceği biçimde değildir. Önce soyut bir yapıya çevrilir. Bu çevirme işlemine problem temsili denir.

Temsil kurmak üç seçim yapmaktır: hangi varlıklar modele girecek, aralarındaki ilişkiler nasıl yazılacak, neye “iyi çözüm” denecek. Modele girmeyen her şey problemin dışında bırakılmış olur. Bu bir kayıp değil, işin kendisidir; ama neyin dışarıda kaldığı bilinmiyorsa sonradan sorun çıkarır.

Üç temel temsil biçimi

Graf temsili

Varlıklar düğüm (*node*), aralarındaki ilişkiler kenar (*edge*) olur. Kenarlara sayısal ağırlık verilebilir.

“İstanbul’dan Ankara’ya en kısa sürede gitmek” problemi şöyle temsil edilir:

- Şehirler düğüm olur.
- Yollar kenar olur.
- Yolun süresi kenarın ağırlığı olur.
- İstanbul başlangıç, Ankara hedef düğümüdür.

Temsil kurulduğu anda problem bir en kısa yol problemine (*shortest path problem*) dönüşür ve Dijkstra’nın algoritması gibi bilinen yöntemler kullanılabilir hale gelir (Dijkstra, 1959).

Durum uzayı temsili

Problem, olası durumların kümesi ve bu durumlar arasındaki geçişler olarak yazılır. Dört bileşeni vardır: başlangıç durumu, uygulanabilir eylemler, eylemin durumu nasıl değiştirdiği, hedef durumun tanımı.

Bir satranç programında:

- Durum, tahtanın o anki dizilişidir.
- Eylem, yapılabilecek bir hamledir.
- Geçiş, hamle sonrası oluşan yeni diziliştir.
- Hedef, şah mat durumudur.

Bu temsil kurulduğunda problem bir arama problemine (*search problem*) dönüşür. Kavramın yapay zekâ alanındaki temel formülasyonu Newell ve Simon tarafından yapılmıştır (Newell ve Simon, 1972).

Kısıt temsili

Problem; deęişkenler, bu deęişkenlerin alabileceęi deęerler ve uyulması gereken kurallar olarak yazılır. Buna en iyilenecek bir hedef fonksiyonu eklenebilir.

Çalışanları vardiyalara dağıtma problemi için:

- Deęişken: her çalışanın hangi vardiyaya atandığı.
- Kısıt: bir çalışan aynı anda iki vardiyada olamaz.
- Kısıt: her vardiyada en az üç kiři bulunmalıdır.
- Hedef: toplam maliyet en küçük olmalıdır.

Temsil kurulduğunda problem bir kısıtlı optimizasyon problemine (*constrained optimization problem*) dönüşür.

Temsilin belirleyicilięi

Temsil, problemin bir kopyası deęildir; bir yorumudur. Bu yüzden temsil deęiştğinde problemin zorluğu da deęişir.

Amarel, aynı bulmacanın farklı temsillerini yan yana koyarak bunu göstermiştir: temsil uygun seçildiğinde arama uzayı küçülür ve problem, hiçbir yeni algoritma bulunmadan, yalnızca yeniden yazıldığı için çözülebilir hale gelir (Amarel, 1968).

Buradan çıkan ilke şudur:

Problemin nasıl temsil edildięi, hangi çözümlerin görülebildiğini belirler.

Bir problemi graf olarak yazan mühendisin aklına graf algoritmaları gelir. Aynı problemi deęişken ve kısıt olarak yazan mühendisin aklına kısıt çözücileri gelir. İkisi de aynı gerçek duruma bakmaktadır.

Sınıflandırma Eksenleri

Problem sınıfları tek bir listede toplanamaz, çünkü farklı sorulara cevap veren birbirinden bağımsız eksenler vardır. Bir problem bu eksenlerin hepsinde aynı anda bir yere düşer.

Amaca göre

Problemin ne istediğini belirtir.

- Arama (*search*): Geçerli bir çözüm bul.
- Karar (*decision*): Evet ya da hayır cevabı üret. Bu terim karmaşıklık teorisinde teknik anlamda kullanılır; çıktısı yalnızca evet veya hayır olan problem demektir. Mühendisin bir tercih yapması anlamına gelmez.
- Optimizasyon (*optimization*): Geçerli çözümler arasından en iyisini bul.
- Kestirim (*estimation*): Var olan ama doğrudan gözlenemeyen bir değeri, gözlenen verilerden çıkar.
- Tahmin (*prediction*): Henüz gerçekleşmemiş bir değeri öngör.

Kestirim ile tahmin ayrımı önemlidir. Bir uydunun şu anki konumunu gürültülü ölçümlerden çıkarmak kestirimdir; yarınki konumunu söylemek tahmindir.

Yapıya göre

Problemin matematiksel iskeletini belirtir.

- Çizelgeleme (*scheduling*): İşleri zamana ve kaynaklara dağıt.
- Rotalama (*routing*): Bir ağ üzerinde hangi güzergâhın izleneceğini belirle.
- Atama (*assignment*): İki kümenin elemanlarını birbiriyle eşleştir.
- Kısıt tatmini (*constraint satisfaction*): Bütün kuralları aynı anda sağlayan bir atama bul.
- Paketleme (*packing*): Sınırlı kapasiteye en değerli içeriği sığdır.
- Kümeleme (*clustering*): Benzer olanları gruplara ayır.
- Akış (*flow*): Kapasiteli bir ağ üzerinde miktarları dağıt.

Bu ailelerin çoğu belirli tarihsel problemlerden doğmuştur. Araç rotalama problemi, yakıt dağıtım kamyonlarının güzergâhlarını belirleme ihtiyacından formüle edilmiştir (Dantzig ve Ramser, 1959).

Alana göre

Aynı yapısal sınıf, farklı mühendislik alanlarında farklı adlarla anılır ve alanın kendi problem aileleri vardır.

- Kontrol problemi: Bir sistemi istenen davranışa yönlendirme.
- Durum kestirimi: Sistemin gerçek durumunu ölçümlerden çıkarma.
- Eşzamanlılık problemi (*concurrency*): Aynı anda çalışan işlerin birbirini bozmasını engelleme.
- Tutarlılık problemi (*consistency*): Birden çok kopyanın aynı veriyi göstermesini sağlama.
- Hata toleransı (*fault tolerance*): Bir parça bozulduğunda sistemin çalışmaya devam etmesi.

Eksenlerin karıştırılması

Sık yapılan hata, farklı eksenlerden gelen etiketleri tek bir listede toplamaktır. “Çizelgeleme, optimizasyon, graf problemi ve NP-zor” yan yana yazıldığında dört ayrı sorunun cevabı tek bir sınıflandırma sanılır. Oysa:

- Çizelgeleme, yapı eksenindedir.
- Optimizasyon, amaç eksenindedir.
- Graf, temsil biçimidir; sınıf değildir.
- NP-zor, hesaplama karmaşıklığı eksenindedir.

Doğru ifade bunları ayırır: “Bu, graf olarak temsil edilen, NP-zor bir çizelgeleme optimizasyonu problemidir.”

Hesaplama Karmaşıklığı Sınıfları

Bu eksen, problemin ne kadar zor olduğunu değil, çözüm süresinin problem büyüdükçe nasıl arttığını sınıflandırır.

Temel sınıflar

- **P**: Girdi boyutunun bir polinomu kadar sürede çözülebilen karar problemleri. Pratikte “makul sürede çözülebilir” anlamına gelir.
- **NP**: Çözümü bulmak zor olabilen, ancak verilen bir çözümün doğruluğu polinom sürede kontrol edilebilen karar problemleri.
- **NP-tam** (*NP-complete*): NP içindeki en zor problemler. Biri polinom sürede çözümlerse hepsi çözülür.
- **NP-zor** (*NP-hard*): En az NP-tam problemler kadar zor olan problemler. Karar problemi olmak zorunda değildirler; optimizasyon problemleri genellikle bu gruba girer.

Sınıfın kurucu sonucu Cook tarafından verilmiş, hemen ardından Karp yirmi bir yaygın kombinatoriyal problemin aynı zorluk sınıfına indirgenebildiğini göstermiştir (Cook, 1971; Karp, 1972). Garey ve Johnson’ın kitabı bu problemlerin geniş bir kataloğunu içerir ve alanda hâlâ başvuru kaynağıdır (Garey ve Johnson, 1979).

Sonucun yanlış okunması

Yaygın çıkarım şudur: “Problem NP-zor, o halde kesin çözüm aranmaz, sezgisel yöntem kullanılır.” Bu çıkarım üç noktada aceledir.

- NP-zorluk, problem ailesinin tamamı için geçerlidir; eldeki belirli örnek için değil. Gerçek verilerde çoğunlukla yapı bulunur ve bu yapı problemi kolaylaştırır.
- NP-zorluk en kötü durumu tarif eder. Ortalama durumda davranış çok farklı olabilir.
- Girdi küçükse kesin çözüm zaten yeterince hızlıdır. Modern tamsayı programlama çözücüleri, on yıl öncesine göre çok daha büyük örnekleri optimum çözmektedir.

Doğru kullanım şudur: NP-zorluk, kesin çözümü yasaklayan bir sonuç değil, kesin çözümün ölçekle birlikte pahalılaşacağını bildiren bir uyarıdır.

Temsilden Algoritmaya Katmanlı Yol

Gerçek dünyadaki bir ihtiyaçtan çalışan koda giden yol, arada birkaç ayrı karar içerir. Bu kararlar ayrı ayrı adlandırıldığında hatanın hangi katmanda yapıldığı görülebilir hale gelir.

Katmanlar

1. Gerçek dünya ihtiyacı: Ne isteniyor.
2. Temsil: İhtiyaç hangi soyut yapıyla yazılıyor.
3. Problem sınıfı: Bu yapı hangi bilinen aileye ait.
4. Çözüm paradigması: Bu aile hangi genel yaklaşımlarla çözülür. Açgözlü yöntem, dinamik programlama, dallandır ve sınırla, doğrusal programlama, kısıt programlama, sezgisel yöntem gibi.
5. Algoritma: Paradigma içinden seçilen belirli yöntem.
6. Uygulama: Yöntemin belirli bir dilde kodlanması.

Örnek

Yirmi kamyonun beş yüz teslimat noktasına dağıtılması problemi katmanlara şöyle oturur:

- İhtiyaç: Teslimatlar en az toplam maliyetle yapılsın.
- Temsil: Noktalar düğüm, yollar kenar, mesafeler ağırlık, kamyon kapasiteleri kısıt.
- Sınıf: Araç rotalama problemi; kombinatorial optimizasyon ailesinde, NP-zor.
- Paradigma: Ölçek küçükse tam yöntem, büyükse sezgisel yöntem.
- Algoritma: Tamsayı programlama modeli ya da yerel arama tabanlı bir sezgisel.
- Uygulama: Seçilen çözücünün arayüzüne modelin yazılması.

Yolun ters yönde kullanılması

Zincir, mevcut kodu değerlendirmek için ters yönde de işletilir. Bütün olası eşleştirmeleri tek tek deneyip en iyisini saklayan bir döngü, önce kaba kuvvet optimizasyonu olarak tanınır; ardından yapının bir atama problemi olduğu görülür. Atama probleminin polinom zamanlı kesin çözümü ise 1955'ten beri bilinmektedir (Kuhn, 1955). Böylece kodun yanlışlığı, satır satır incelenerek değil, ait olduğu sınıf tanınarak ortaya çıkar.

Akışın tek yönlü olmaması

Katmanlar sıralı görünse de akış tek yönlü değildir. İki yaygın geri dönüş vardır.

Birincisi, alt katmanda çıkan tıkanıklıktır: seçilen algoritma kabul edilebilir sürede sonuç vermediğinde model gevşetilir, bazı kısıtlar yumuşatılır ya da problem parçalara bölünür. İkincisi ve daha sık görüleni, alt katmanın üst katmanı baştan biçimlendirmesidir: kurumda halihazırda kullanılan bir çözücü varsa model, o çözücünün kabul ettiği biçimde kurulur. Bu, kendiliğinden bir hata değildir; ancak modelin gerçeği değil aracı yansıttığı durumlarda hata kaynağı olur.

Taksonomi, Ontoloji ve Temsil

Üç terim yakın durur ve farklı işler yapar.

Taksonomi

Taksonomi, nesnelerin hiyerarşik gruplara ayrılmasıdır. Cevapladığı soru şudur: neyin altında ne var. Biyolojideki canlı sınıflandırması bunun bilinen örneğidir. Problem taksonomisi de benzer biçimde problem ailelerini üst ve alt kümelere yerleştirir; en kısa yol probleminin graf problemlerinin altında yer alması gibi.

Ontoloji

Ontoloji, bir alandaki kavramların ve aralarındaki ilişkilerin açık biçimde tanımlanmasıdır. Cevapladığı soru şudur: bu alanda hangi kavramlar vardır ve birbirleriyle nasıl ilişkilendirilir. Bilgi sistemleri bağlamında yerleşik tanım, ontolojiyi bir kavram-sallaştırmanın açık belirtimi olarak verir (Gruber, 1993). Ontoloji, taksonomiden genişler; hiyerarşi ontolojinin yalnızca bir ilişki türüdür.

Temsil

Temsil, taksonomi ve ontolojiden farklı bir düzlemde. İkisi bir alan hakkında konuşurken temsil belirli bir problem hakkında konuşur. Cevapladığı soru şudur: elimdeki bu problemi hangi yapıyla yazacağım.

Ayrım kısaca şöyledir: taksonomi neyin hangi kutuya girdiğini söyler, ontoloji kutuların ne olduğunu ve nasıl ilişkilendiğini söyler, temsil ise belirli bir problemin hangi biçimde yazıldığını söyler.

Çerçevenin Sınırları

Problemler sınıflandırılmaya hazır gelmez

Ders kitabı örnekleri “bin iş, yirmi makine, toplam süreyi en küçükle” biçiminde başlar. Sahadaki problem “üretim gecikiyor” biçiminde başlar. İkisi arasındaki mesafe, sınıflandırmanın kendisinden daha fazla emek ister.

Simon bu ayrımı yapılandırılmış ve yapılandırılmamış problem (*well-structured* ve *ill-structured problem*) olarak adlandırmıştır. Yapılandırılmamış problemde amacın ne olduğu, hangi bilginin ilgili olduğu ve çözümün ne zaman kabul edileceği baştan belli değildir (Simon, 1973). Sınıflandırma çerçevesi ancak problem yapılandırıldıktan sonra devreye girer.

Planlama ve politika alanında bunun daha uç bir hali tanımlanmıştır. Rittel ve Webber, problem tanımının çözüm önerisinden bağımsız olmadığı, doğru ya da yanlış cevabın bulunmadığı durumları belirsiz problem (*wicked problem*) olarak adlandırır (Rittel ve Webber, 1973). Bu tür problemlerde sınıflandırma çabası yanıltıcıdır, çünkü sınıflandırılan şey problemin kendisi değil onun geçici bir yorumudur.

Yanlış model, doğru algoritma

Katmanlı yapının en pahalı hatası algoritma seçiminde değil temsil katmanında oluşur. Yanlış kurulmuş bir model üzerinde doğru çalışan bir algoritma, kendinden emin bir yanlış üretir. Hata gürültülü değildir; tutarlıdır ve bu yüzden geç fark edilir.

Örneğin teslimat probleminde kamyon kapasitesi modele girmemişse çözücü optimum bir rota üretir ve bu rota uygulanabilir değildir. Çözücüde hata yoktur, çıktı da matematiksel olarak doğrudur; temsil eksiktir.

Sınıf bilgisinin yöntem garantisi vermemesi

Problemin sınıfını bilmek hangi yöntemin iyi çalışacağını garanti etmez. Wolpert ve Macready, bütün olası problemler üzerinde ortalama alındığında hiçbir optimizasyon yönteminin diğerinden üstün olmadığını göstermiştir (Wolpert ve Macready, 1997). Bir yöntemin üstünlüğü her zaman belirli bir problem yapısına bağlıdır. Dolayısıyla sınıf bilgisi, yöntem seçimini daraltır ama yerine geçmez; deneysel karşılaştırma ortadan kalkmaz.

Sonuç

Problem sınıflandırması, çözümüazardan bulma yöntemi değil, arama alanını daraltma yöntemidir. Sağladığı kazanç, bilinen bir aileye ait olduğu anlaşılan problem için o ailenin birikmiş yöntemlerinin, zorluk sonuçlarının ve araçlarının devralınabilmesidir.

Pratik karşılığı üç noktada toplanır. Birincisi, sınıf etiketleri eksen belirtilerek kullanılmalıdır; amaç, yapı, temsil ve karmaşıklık ayrı sorulardır ve tek bir listede toplandıklarında birbirini gizlerler. İkincisi, NP-zorluk bir yasak değil bir ölçek uyarısıdır ve eldeki örneğin boyutu ile yapısı görülmeden sezgisel yonteme geçilmesini gerekçelendirmez. Üçüncüsü, hata arandığında önce temsil katmanına bakılmalıdır; algoritma katmanındaki hatalar gürültülü ve erken görünür, temsil katmanındakiler sessiz ve tutarlıdır.

Çerçevenin çalışmadığı yer, problemin henüz yapılandırılmamış olduğu durumdur. Amacın, ilgili bilginin ve kabul ölçütünün belirsiz olduğu bir problemde sınıflandırma erken bir kapanış üretir: belirsizlik çözülmez, yalnızca bir etiketin altına saklanır.

Açık kalan soru, bir problemin hangi temsille yazılacağına neyin karar verdiğidir. Temsil seçiminin sonuçları bilinmektedir, ancak seçimin kendisi büyük ölçüde deneyime dayanır ve sistematik bir yöntemi yoktur. Amarel'den bu yana temsil değiştirme (*representation change*) üzerine çalışılmıştır, fakat pratikte bu hâlâ öğretilebilir bir prosedür değil, tanımlanabilir bir ustalıktır.

Kaynaklar

- Amarel, S. (1968). On Representations of Problems of Reasoning about Actions. *Machine Intelligence*, 3.
- Cook, S. A. (1971). The Complexity of Theorem-Proving Procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*.
- Dantzig, G. B. ve Ramser, J. H. (1959). The Truck Dispatching Problem. *Management Science*, 6(1).
- Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1.
- Garey, M. R. ve Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition*, 5(2).
- Karp, R. M. (1972). Reducibility Among Combinatorial Problems. *Complexity of Computer Computations*. Plenum Press.
- Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2.

- Newell, A. ve Simon, H. A. (1972). *Human Problem Solving*. Prentice-Hall.
- Rittel, H. W. J. ve Webber, M. M. (1973). Dilemmas in a General Theory of Planning. *Policy Sciences*, 4(2).
- Russell, S. ve Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. 4. baskı. Pearson.
- Simon, H. A. (1973). The Structure of Ill-Structured Problems. *Artificial Intelligence*, 4(3-4).
- Wolpert, D. H. ve Macready, W. G. (1997). No Free Lunch Theorems for Optimization. *IEEE Transactions on Evolutionary Computation*, 1(1).